CS342 · VISUAL PROGRAMMING

Chapter 2 Graphical Interfaces

ملخص الميد

محارب

محتوى الملف

01 AWT و Swing

02 Containers و Components

03 النافذة JFrame

04 Content pane و JPanel

05 Layout managers

06 الأحداث Events

07 Adapters

08 Dialogs و JOptionPane

01 AWT و Swing

Java offers 2 systems for creating graphical interfaces: the oldest is **AWT** (Abstract Window Toolkit, package java.awt); the most recent (since Java 2) is **Swing**, an improvement of AWT (package javax.swing). Swing is a set of classes used to create objects representing graphic components.

	AWT	Swing
Package	**java.awt**	**javax.swing**
النوع	Heavyweight	Lightweight
يعتمد على	مدير النوافذ في نظام التشغيل، ويستهلك موارد أكثر	مستقل عن النظام: يرسم العنصر بنفسه
الشكل	يختلف حسب النظام	نفس الشكل في كل مكان

- AWT ما فيه إلا مكونات ثقيلة، وياخذ بس الأشياء المشتركة بين كل الأنظمة (Least Common Denominator).
- كل مكونات Swing خفيفة **إلا JApplet** و **JDialog** و **JFrame** و **JWindow**.
- أسماء مكونات Swing تبدأ بـ J: **JButton** مقابل **Button** في AWT.

02 Components و Containers

A graphical interface in Java is an assembly of **containers** (Container) and **components** (Component).

• A component is a "visible" part of the user interface (buttons, text, drawing areas). It is a subclass of java.awt.Component.

• A container is a space in which you can position several components (windows, applets). Container is itself a subclass of Component.

Container	وش هو
Frame / JFrame	نافذة رئيسية فيها عنوان وإطار وتقدر تكبرها وتصغرها
Panel / JPanel	ما له شكل، وما ينفع نافذة لحاله. تحطه داخل نافذة زي أي زر، وله layout خاص فيه

المجموعة	أمثلة
مكونات أساسية	JButton, JCheckBox, JRadioButton, JToggleButton, JComboBox, JList, JSlider, JTable, JTree, JProgressBar, JSpinner
القوائم	JMenuBar, JMenu, JMenuItem, JCheckBoxMenuItem, JRadioButtonMenuItem, JPopupMenu, JToolBar, JToolTip
النصوص	JTextField, JPasswordField, JTextArea, JTextPane, JEditorPane
Containers	JFrame, JDialog, JOptionPane, JPanel, JScrollPane, JTabbedPane, JSplitPane, JInternalFrame, JDesktopPane, JWindow

مكون النص	يستخدم لـ
JTextField	سطر واحد
JPasswordField	سطر واحد ويخفي الحروف
JTextArea	أكثر من سطر
JEditorPane	أكثر من سطر وأنواع محتوى مختلفة: نص وHTML
JLabel	نص ثابت للعرض بس

03 النافذة JFrame

الدالة	وش تسوي
show() / setVisible(true)	تعرض النافذة. بدونها ما يطلع شي
setSize(w, h)	الحجم. الافتراضي صفر
setTitle("...")	النص اللي في شريط العنوان
setLocation(x, y)	مكان النافذة على الشاشة
setBounds(x, y, w, h)	المكان والحجم مرة وحدة
pack()	تصغّر النافذة لأقل حجم يبين كل المكونات
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)	زر الإغلاق يوقف البرنامج
setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE)	يقفل النافذة بس والبرنامج يكمل

Warning: by default, closing a JFrame does not terminate the program, but simply makes the window invisible (as if calling setVisible(false)).

عشان النافذة تتفاعل مع المستخدم، نسوي كلاس خاص يورث من **JFrame**، ونكتب كل شي في الـ constructor:

```
import javax.swing.*;
public class Window extends JFrame {
    public Window() {
        setTitle("My first window");
        setSize(300, 150);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public static void main(String[] args) {
        Window premFen = new Window();
        premFen.setVisible(true);
        premFen.setBounds(250, 250, 300, 200);
    }
}
```

مكان النافذة حسب حجم الشاشة يحتاج **Toolkit** من **java.awt**:

```
Toolkit kit = Toolkit.getDefaultToolkit();
Dimension screenSize = kit.getScreenSize();
int H = screenSize.height;
int W = screenSize.width;
Image img = kit.getImage("icon.gif");
this.setIconImage(img);          // change the window icon
```

04 JPanel و Content pane

JFrame has a complex structure (a root, a content, etc.). Components are added to the **content** part with the method **add**. getContentPane() provides the reference to this content of type Container.

```
Container c = getContentPane();
JPanel c = (JPanel) getContentPane();     // or cast it to JPanel
```

المشكلة: لو أضفت أكثر من مكون بدون layout، **ما يطلع إلا آخر واحد**، لأن **add** تحطهم كلهم في نفس المكان. الحل: تقسم النافذة بـ layout.

Absolute positioning: تلغي الـ layout وتحدد مكان وحجم كل مكون بنفسك.

```
c.setLayout(null);
button1.setBounds(x, y, width, height);   // x from the left, y from the top
```

05 Layout managers

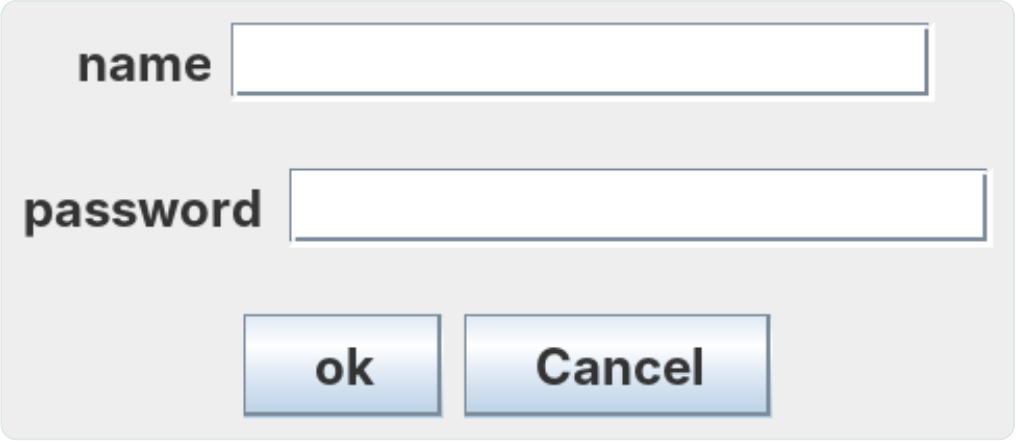
Each container is associated with a layout manager, which controls the location and size of components each time they are displayed. Every container has a default one; change it with setLayout().

Container	الـ layout الافتراضي
JFrame	BorderLayout
JPanel	FlowLayout

Layout	كيف يرتب	الكتابة
FlowLayout	صف ورا صف، من اليسار لليمين، وإذا امتلأ الصف ينزل للي بعده. كل مكون بحجمه الطبيعي (preferredSize). الافتراضي في النص	**new FlowLayout(FlowLayout.LEFT)**
BorderLayout	5 مناطق: NORTH و SOUTH و EAST و WEST و CENTER، ومكون واحد بس في كل منطقة	**add(b, BorderLayout.NORTH)**
GridLayout	شبكة صفوف وأعمدة، وكل الخانات بنفس الحجم، والمكون يعبي خانته	**new GridLayout(3, 2)**

- **FlowLayout:** يخفي المكونات اللي ما تكفيها النافذة، ويناسب لما تكون المكونات قليلة. المحاذاة: **LEFT** أو **RIGHT** أو **CENTER**.
- **BorderLayout:** الشمال والجنوب ياخذون طولهم الطبيعي وعرض النافذة كامل. الشرق والغرب ياخذون عرضهم الطبيعي والطول اللي باقي. الـ CENTER ياخذ كل اللي باقي. والمنطقة الفاضية حجمها صفر.
- **GridLayout:** تقدر تخلي عدد الصفوف أو الأعمدة صفر، بس مو الاثنين. المسافات بين الخانات بـ **setHgap()** و **setVgap()**، والافتراضي صفر. يناسب لوحة أرقام أو toolbar.
- **تركيب الـ layouts:** النافذة نفسها **GridLayout** أو **BorderLayout**، وكل خانة فيها **JPanel** له **FlowLayout**. هذي الطريقة اللي تطلع أي شكل يجيك في الميد.

```
Container p = getContentPane();
p.setLayout(new GridLayout(3, 1));                          // 3 rows
JPanel p1 = new JPanel(new FlowLayout(FlowLayout.CENTER));  // row 1
JPanel p2 = new JPanel(new FlowLayout(FlowLayout.LEFT));    // row 2
JPanel p3 = new JPanel(new FlowLayout(FlowLayout.CENTER));  // row 3
p1.add(l1); p1.add(t1);
p2.add(l2); p2.add(t2);
p3.add(b1); p3.add(b2);
p.add(p1); p.add(p2); p.add(p3);
pack();
```



06 Events الأحداث

When the user performs an action (mouse click, item selection, keystroke), an **event** is issued. The component emitting it is the **source**; it transmits the event to one or more **listeners**.

To process an event of type XXXEvent, a listener must implement the interface XXXListener, and must register with the component using addXXXListener(). A component can be its own listener.

المصدر	Event	Listener	الدالة
ضغط زر، Enter في خانة نص	**ActionEvent**	**ActionListener**	**actionPerformed(ActionEvent e)**
تغير قيمة خانة نص	**TextEvent**	**TextListener**	**textValueChanged(TextEvent e)**
اختيار عنصر من قائمة	**ItemEvent**	**ItemListener**	**itemStateChanged(ItemEvent e)**
الـ focus يجي أو يروح	**FocusEvent**	**FocusListener**	**focusLost** · **focusGained**
الماوس	**MouseEvent**	**MouseListener**	5 دوال: mouseClicked و mouseEntered و mouseExited و mousePressed و mouseReleased

طريقة 1: النافذة نفسها هي الـ listener. الكلاس يسوي **implements ActionListener**، وتسجّل بـ **this**. لو عندك أكثر من زر، تفرق بينهم بـ **e.getSource()**.

```
public class Window7 extends JFrame implements ActionListener {
    private JButton b1 = new JButton("OK");
    private JButton b2 = new JButton("To leave");
    private JTextField t = new JTextField();
    public Window7() {
        Container c = this.getContentPane();
        c.setLayout(new BorderLayout());
        c.add(b1, BorderLayout.CENTER);
        c.add(b2, BorderLayout.SOUTH);
        c.add(t, BorderLayout.NORTH);
        b1.addActionListener(this);      // the 2 buttons share one listener
        b2.addActionListener(this);
    }
    public void actionPerformed(ActionEvent e) {
        if ((JButton) e.getSource() == b1) t.setText("OK button pressed");
        else t.setText("EXIT button pressed");
    }
}
```

طريقة 2: inner class لكل زر. كلاس داخل كلاس النافذة، يقدر يوصل لمتغيراتها مباشرة. هذي الطريقة اللي يطلبها الميد (BConvert و BCancel).

```
b1.addActionListener(new OkListener());      // in the constructor
class OkListener implements ActionListener {  // inside the window class
    public void actionPerformed(ActionEvent e) {
        t.setText("OK button pressed");
    }
}
```

طريقة 3: anonymous class. نفس الـ inner class بس بدون اسم، وتنكتب داخل **add...Listener** مباشرة.

```
addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        String s = "";
        s += ++nb;
        l2.setText(s);
    }
});
```

The method **Object getSource()** is present in all event classes and returns the source object of the event. getX() and getY() on a MouseEvent return the mouse coordinates.

Adapters 07

A class that implements an interface must redefine **all** its methods. Some listener interfaces have many methods, so Java offers **Adapter** classes: inherit from the adapter and redefine only the methods you need. Naming principle: XXXAdapter corresponds to XXXListener.

	implements MouseListener	extends MouseAdapter
الكلمة	implements	extends
تكتب	الـ 5 دوال كلها، حتى اللي ما تحتاجها تكتبها فاضية	بس الدالة اللي تبيها

مثال السلايد: نافذة تعد ضغطات الماوس. بـ **MouseListener** تكتب **mouseClicked** وتخلي الأربع الباقية فاضية، وبـ **MouseAdapter** تكتب **mouseClicked** بس.

وتقفل البرنامج من زر بـ **System.exit(0);** داخل **actionPerformed**.

JOptionPane و Dialogs 08

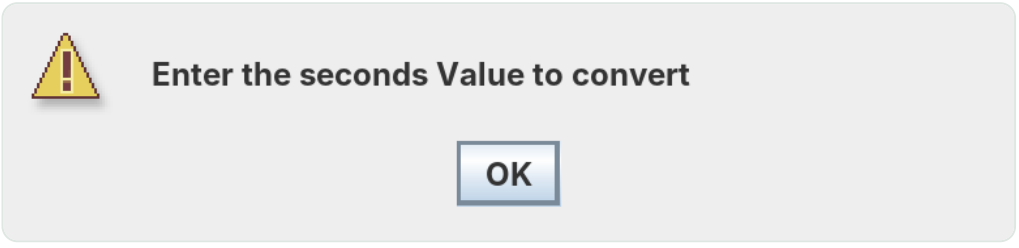
A dialog is a container that you make appear or disappear. Java provides standard dialog boxes through the class **JOptionPane**: message, confirmation, input and option boxes. The class JDialog builds custom dialog boxes.

```
JOptionPane.showMessageDialog(null, "Hello");
```

```
JOptionPane.showMessageDialog(null, "warning", "box title",
JOptionPane.WARNING_MESSAGE);
```

```
//                          parent  message    title        icon type
```

نوع الأيقونة	الأيقونة
JOptionPane.ERROR_MESSAGE	خطأ ✕
JOptionPane.INFORMATION_MESSAGE	معلومة i
JOptionPane.WARNING_MESSAGE	تحذير ⚠
JOptionPane.QUESTION_MESSAGE	سؤال ?
JOptionPane.PLAIN_MESSAGE	بدون أيقونة



الصندوق	الدالة	يرجع
رسالة	**showMessageDialog**	ما يرجع شي
إدخال	**showInputDialog(null, "Give a text")**	**String** اللي كتبه، أو **null** إذا ما ضغط OK
خيارات	**showInputDialog(null, msg, title, type, null, colors, colors[1])**	**Object** اللي اختاره من القائمة، أو **null**
تأكيد	**showConfirmDialog(null, "Do you want to continue?")**	**int** حسب الزر

أزرار التأكيد	القيمة اللي ترجع
DEFAULT_OPTION · **YES_NO_OPTION** · **YES_NO_CANCEL_OPTION** · **OK_CANCEL_OPTION**	**YES_OPTION** (0) · **OK_OPTION** (0) · **NO_OPTION** (1) · **CANCEL_OPTION** (2) · **CLOSED_OPTION** (−1)

```
int n = JOptionPane.showConfirmDialog(null, "Do you want to continue?",
                                      "Major incident", JOptionPane.YES_NO_OPTION);
```

```
String name = JOptionPane.showInputDialog(null, "Enter your name");
```

```
int a = Integer.parseInt(JOptionPane.showInputDialog(null, "First integer"));
```